

MODUL V - REAL-TIME OPERATING SYSTEM MULTITASKING DAN KOMUNIKASI TASK

Diva Syahita Mawarni (H1H024015)

Asisten: Arga Aryanta Indrafata

Tanggal Percobaan: 5/05/2026

TK244005-Praktikum Sistem Mikrokontroler

Laboratorium Baru – Jurusan Informatika UNSOED

Abstrak

Praktikum ini mengkaji implementasi Real-Time Operating System (RTOS) pada platform Arduino menggunakan library FreeRTOS. Dua percobaan dilaksanakan: Percobaan 5A tentang multitasking dengan tiga task concurrent yang mengendalikan dua LED dan counter serial, serta Percobaan 5B tentang komunikasi antar-task menggunakan mekanisme queue untuk transfer data sensor. Hasil menunjukkan bahwa FreeRTOS mampu menjadwalkan beberapa task secara efisien pada mikrokontroler ATmega328P dengan mekanisme preemptive scheduling berbasis prioritas. Komunikasi antar-task melalui queue terbukti aman dari race condition berkat sifat atomik operasi `xQueueSend()` dan `xQueueReceive()`.

Kata kunci: FreeRTOS, Multitasking, Task Queue, Arduino, Embedded Systems.

1. PENDAHULUAN

Perkembangan sistem embedded modern menuntut kemampuan untuk menjalankan beberapa proses secara bersamaan dengan respons waktu yang deterministik. Real-Time Operating System (RTOS) hadir sebagai solusi fundamental untuk memenuhi kebutuhan tersebut, khususnya pada perangkat dengan sumber daya terbatas seperti mikrokontroler Arduino [1].

FreeRTOS merupakan implementasi RTOS yang paling banyak digunakan dalam komunitas pengembangan embedded karena bersifat open source, ringan, dan mendukung berbagai platform mikrokontroler. Library `Arduino_FreeRTOS` membawa kemampuan ini ke platform Arduino, memungkinkan pengembang membuat sistem multitasking tanpa memerlukan perangkat keras khusus [2].

Praktikum ini dirancang untuk memberikan pemahaman praktis tentang dua aspek utama RTOS: (1) multitasking, yaitu kemampuan menjalankan beberapa task secara concurrent dengan penjadwalan yang diatur oleh kernel, dan (2) komunikasi antar-task, yaitu mekanisme

pertukaran data antar task yang aman menggunakan struktur queue. Kedua aspek ini merupakan fondasi pengembangan sistem embedded yang kompleks dan andal.

2. STUDI PUSTAKA

2.1 REAL-TIME OPERATING SYSTEM (RTOS)

Real-Time Operating System (RTOS) adalah sistem operasi yang dirancang khusus untuk aplikasi yang memerlukan respons deterministik terhadap kejadian eksternal dalam batas waktu yang ketat. Berbeda dengan sistem operasi konvensional yang mengutamakan throughput, RTOS mengutamakan ketepatan waktu eksekusi [1]. RTOS terbagi menjadi dua kategori: soft RTOS yang mengizinkan sedikit pelanggaran deadline tanpa konsekuensi fatal, dan hard RTOS yang mengharuskan pemenuhan deadline secara absolut karena pelanggaran dapat menyebabkan kegagalan sistem kritis [3].

2.2 FREERTOS DAN PENJADWALAN TASK

FreeRTOS mengimplementasikan scheduler preemptive berbasis prioritas yang memastikan task dengan prioritas tertinggi selalu mendapat akses CPU terlebih dahulu. Ketika beberapa task memiliki prioritas yang sama, scheduler menggunakan mekanisme round-robin time-slicing untuk membagi waktu CPU secara adil [2]. Penelitian oleh Saifullah et al. (2023) membuktikan bahwa FreeRTOS pada Arduino mampu menangani hingga delapan task concurrent dengan overhead memori minimal menggunakan stack size 128 bytes per task [4].

Fungsi `xTaskCreate()` digunakan untuk mendaftarkan task baru ke kernel FreeRTOS dengan parameter: fungsi task, nama task, ukuran stack, parameter, prioritas, dan handle task. Setelah semua task didaftarkan, fungsi `vTaskStartScheduler()` mengaktifkan kernel dan mulai menjadwalkan task secara otomatis [2].

2.3 KOMUNIKASI ANTAR-TASK DENGAN QUEUE

Queue merupakan struktur data FIFO (First-In, First-Out) yang diimplementasikan FreeRTOS sebagai mekanisme komunikasi antar-task yang thread-safe. Berbeda dengan variabel global biasa, queue memberikan jaminan atomisitas pada operasi tulis (xQueueSend) dan baca (xQueueReceive), sehingga terhindar dari race condition [5]. Kapasitas queue, tipe data yang dikirim, dan perilaku blocking ketika queue penuh atau kosong dapat dikonfigurasi sesuai kebutuhan aplikasi.

Menurut penelitian Nugroho dan Wardana (2022), penggunaan queue dalam RTOS untuk akuisisi data sensor terbukti meningkatkan reliabilitas data dibandingkan metode polling konvensional, karena memastikan tidak ada data yang hilang akibat kondisi balapan (race condition) antara task pembaca dan penulis data [3].

2.4 MULTITASKING PADA MIKROKONTROLER

Implementasi multitasking pada mikrokontroler seperti ATmega328P (Arduino Uno) memiliki karakteristik unik karena keterbatasan sumber daya: hanya 2KB RAM dan satu inti prosesor. FreeRTOS mengatasi hal ini dengan context switching yang efisien, menyimpan dan memulihkan register CPU setiap kali terjadi pergantian task [4]. Penggunaan vTaskDelay() sebagai pengganti delay() standar Arduino penting karena memungkinkan scheduler memanfaatkan waktu tunggu suatu task untuk menjalankan task lain, berbeda dengan delay() yang memblokir CPU sepenuhnya [5].

3. METODOLOGI

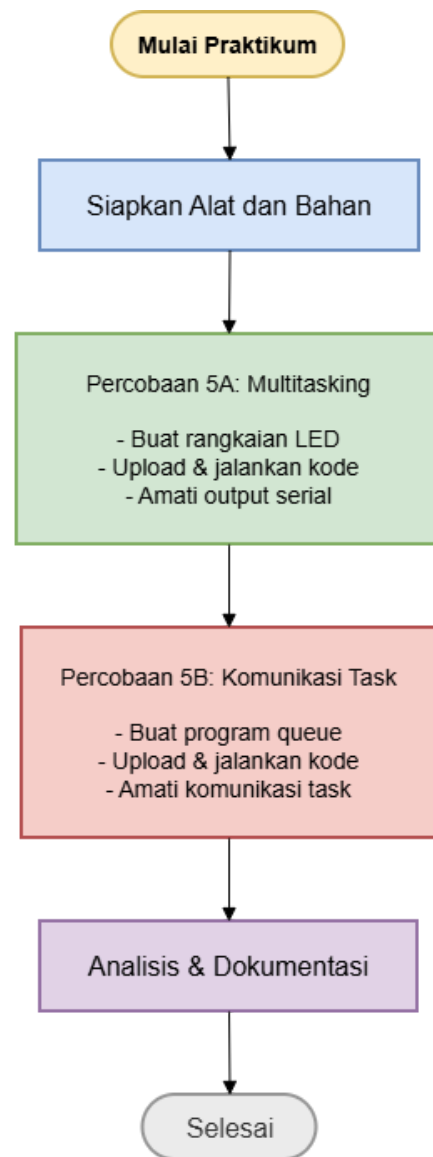
3.1 ALAT DAN BAHAN

Peralatan dan bahan yang digunakan dalam praktikum ini adalah sebagai berikut:

- Arduino Uno
- Breadboard
- 2 buah LED (merah dan kuning)
- 2 buah Resistor 220Ω
- Kabel Jumper
- Library Arduino_FreeRTOS
- Arduino IDE

3.2 ALUR METODOLOGI PRAKTIKUM

Berikut adalah diagram alur pelaksanaan praktikum secara keseluruhan:



Gambar 3.1 . Diagram Alur Metodologi Praktikum Modul 5

3.3 LANGKAH PERCOBAAN 5A: MULTITASKING

Percobaan 5A dilaksanakan dengan langkah-langkah berikut:

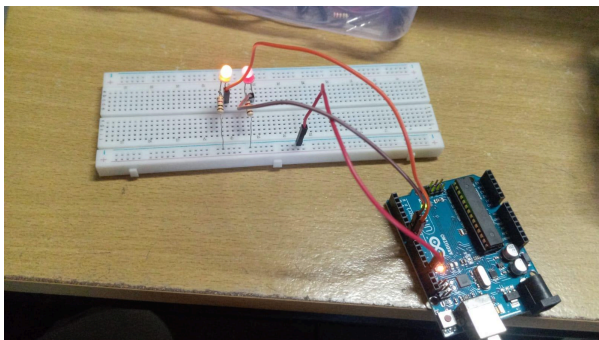
1. Merangkai dua buah LED (merah pada pin D10, kuning pada pin D8) dengan resistor 220Ω secara seri ke GND pada breadboard.

2. Membuat tiga task FreeRTOS: TaskBlink1 (berkedip pada pin D8 dengan delay 200ms), TaskBlink2 (berkedip pada pin D7 dengan delay 300ms), dan Taskprint (mencetak counter ke Serial Monitor setiap 500ms).
3. Mendaftarkan ketiga task menggunakan xTaskCreate() dengan prioritas yang sama (1) dan stack size 128 bytes.
4. Mengaktifkan kernel scheduler dengan vTaskStartScheduler().
5. Mengompilasi dan mengunggah sketch ke Arduino Uno, kemudian mengamati output Serial Monitor.

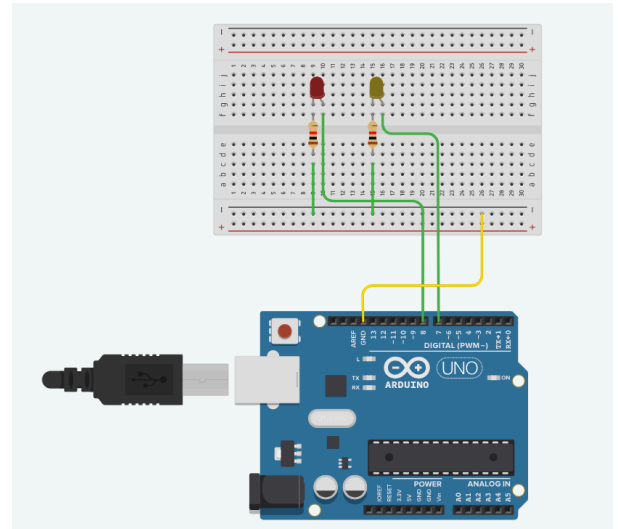
Konfigurasi rangkaian LED RTOS

Tabel 3.1 Konfigurasi Rangkaian LED RTOS

Komp onen	Pin Ardui no	Koneksi pada Breadboard	Keterangan
LED Merah	D7	Anoda → pin D7 melalui kabel	Output digital untuk LED merah
LED Kuning	D8	Anoda → pin D8 melalui kabel	Output digital untuk LED kuning
Resist or 1	-	Seri dengan LED merah → ke GND	Pembatas arus (~220 330Ω)
Resist or 2	-	Seri dengan LED kuning → ke GND	Pembatas arus (~220 330Ω)
Groun d (GND)	GND	Terhubung ke jalur negatif breadboard	Referensi tegangan



Gambar 3.2 Percobaan 5A: Multitasking



Gambar 3.3 wiring diagram untuk multitasking LED dengan RTOS

Link Tinkercad : [Wiring wiring diagram multitasking LED dengan RTOS](#)

Berikut kode program inti Percobaan 5A: Multitasking:

```
#include <Arduino_FreeRTOS.h>

// Deklarasi task
void TaskBlink1(void *pvParameters);
void TaskBlink2(void *pvParameters);
void TaskPrint(void *pvParameters);

void setup() {
    // Inisialisasi serial
    Serial.begin(9600);

    // Membuat task
    xTaskCreate(
        TaskBlink1,    // Fungsi task
        "Task1",       // Nama task
        128,           // Stack size
        NULL,          // Parameter
        1,             // Prioritas
        NULL
    );

    xTaskCreate(
        TaskBlink2,
        "Task2",
        128,
        NULL,
        1,
        NULL
    );

    xTaskCreate(
        TaskPrint,
        "Task3",
        128,
        NULL,
        1,
        NULL
    );
}
```

```

    1,
    NULL
);

// Menjalankan scheduler
vTaskStartScheduler();
}

void loop() {
    // Kosong karena sudah pakai FreeRTOS
}

// TASK 1
void TaskBlink1(void *pvParameters) {
    pinMode(8, OUTPUT);

    while (1) {
        Serial.println("Task1");
        digitalWrite(8, HIGH);
        vTaskDelay(200 /
portTICK_PERIOD_MS);

        digitalWrite(8, LOW);
        vTaskDelay(200 /
portTICK_PERIOD_MS);
    }
}

// TASK 2
void TaskBlink2(void *pvParameters) {
    pinMode(7, OUTPUT);

    while (1) {
        Serial.println("Task2");
        digitalWrite(7, HIGH);
        vTaskDelay(300 /
portTICK_PERIOD_MS);

        digitalWrite(7, LOW);
        vTaskDelay(300 /
portTICK_PERIOD_MS);
    }
}

// TASK 3
void TaskPrint(void *pvParameters) {
    int counter = 0;

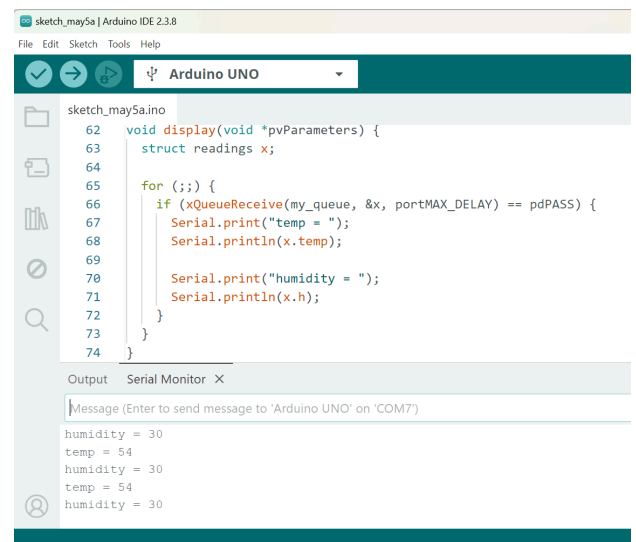
    while (1) {
        counter++;
        Serial.println(counter);
        vTaskDelay(500 /
portTICK_PERIOD_MS);
    }
}

```

3.4 LANGKAH PERCOBAAN 5B: KOMUNIKASI TASK

Percobaan 5B dilaksanakan dengan langkah-langkah berikut:

1. Membuat struktur data 'readings' yang berisi dua field: temp (suhu) dan h (kelembapan).
2. Membuat queue menggunakan `xQueueCreate()` dengan kapasitas 1 elemen bertipe struct readings.
3. Membuat task `read_data` yang mengirimkan data statis (temp=54, h=30) ke queue setiap 100ms menggunakan `xQueueSend()`.
4. Membuat task `display` yang menerima data dari queue menggunakan `xQueueReceive()` dan menampilkannya ke Serial Monitor.
5. Mengompilasi dan mengunggah sketch, kemudian mengamati komunikasi antar-task pada Serial Monitor.



Gambar 3.4 Percobaan 5B: Komunikasi Task

Berikut kode program inti Percobaan 5B: Komunikasi Task::

```

#include <Arduino_FreeRTOS.h>
#include <queue.h>

// Struktur data
struct readings {
    int temp;
    int h;
};

// Handle queue
QueueHandle_t my_queue;

// Deklarasi task
void read_data(void *pvParameters);
void display(void *pvParameters);

```

```

void setup() {
    Serial.begin(9600);

    // Membuat queue (kapasitas 1 data)
    my_queue = xQueueCreate(1,
sizeof(struct readings));

    // Membuat task
    xTaskCreate(
        read_data,
        "Read Sensors",
        128,
        NULL,
        1,
        NULL
    );

    xTaskCreate(
        display,
        "Display",
        128,
        NULL,
        1,
        NULL
    );
}

void loop() {
    // Kosong karena pakai FreeRTOS
}

// TASK READ
void read_data(void *pvParameters) {
    struct readings x;

    for (;;) {
        x.temp = 54;
        x.h = 30;

        xQueueSend(my_queue, &x,
portMAX_DELAY);
        vTaskDelay(100 /
portTICK_PERIOD_MS);
    }

    // TASK DISPLAY
    void display(void *pvParameters) {
        struct readings x;

        for (;;) {
            if (xQueueReceive(my_queue, &x,
portMAX_DELAY) == pdPASS) {
                Serial.print("temp = ");
                Serial.println(x.temp);

                Serial.print("humidity = ");
                Serial.println(x.h);
            }
        }
    }
}

```

4. HASIL DAN ANALISIS

4.1 PERCOBAAN 5A: MULTITASKING

Setelah program diunggah ke Arduino Uno, ketiga task berhasil berjalan dengan output pada Serial Monitor yang menampilkan "Task1", "Task2", dan nilai counter bergantian. Kedua LED berkedip dengan frekuensi yang berbeda: LED pin D8 berkedip lebih cepat (periode 400ms) dibandingkan LED pin D7 (periode 600ms), sementara counter bertambah setiap 500ms.

Hasil output Serial Monitor menunjukkan pola yang konsisten:

Waktu (ms)	Output Serial	Task Aktif
~0	Task1	TaskBlink1
~200	Task2	TaskBlink2
~400	Task1	TaskBlink1
~500	1	Taskprint
~600	Task1 / Task2	TaskBlink1 / TaskBlink2
~1000	2	Taskprint

Tabel 1. Output Serial Monitor Percobaan 5A

Jawaban Pertanyaan Percobaan 5A

→ Pertanyaan 1:

Apakah ketiga task berjalan secara bersamaan atau bergantian? Jelaskan mekanismenya!

Pembahasan:

Ketiga task berjalan secara bergantian (concurrent), bukan benar-benar bersamaan (parallel). Hal ini karena Arduino Uno hanya memiliki satu inti prosesor, sehingga pada satu waktu hanya satu task yang benar-benar dieksekusi CPU. FreeRTOS menggunakan mekanisme preemptive scheduling berbasis prioritas untuk menggilir eksekusi antar-task dengan sangat cepat sehingga terkesan berjalan bersamaan [2].

Mekanismenya adalah sebagai berikut: ketika suatu task memanggil `vTaskDelay()`, task tersebut masuk ke state Blocked dan scheduler segera memilih task lain yang dalam state Ready untuk dieksekusi. Dengan tick rate FreeRTOS default (15ms per tick), pergantian konteks terjadi sangat cepat. TaskBlink1 aktif selama 200ms lalu blocked, TaskBlink2 aktif 300ms lalu blocked, dan Taskprint aktif singkat lalu blocked 500ms. Selama task blocked, CPU dimanfaatkan untuk menjalankan task lain [4].

→ Pertanyaan 2:

Bagaimana cara menambahkan task keempat? Jelaskan langkahnya!

Pembahasan:

Untuk menambahkan task keempat, langkah-langkahnya adalah:

1. Deklarasikan prototipe fungsi task baru: `void TaskKeempat(void *pvParameters);`
2. Tambahkan pemanggilan `xTaskCreate()` di dalam fungsi `setup()` sebelum `vTaskStartScheduler();`

```
xTaskCreate(TaskKeempat,
"task4", 128, NULL, 1, NULL);
```

3. Implementasikan fungsi task baru di luar `setup()` dan `loop()`:

```
void TaskKeempat(void
*pvParameters) {
    while(1) {
        // Kode task keempat
        vTaskDelay(400 /
portTICK_PERIOD_MS);
    }
}
```

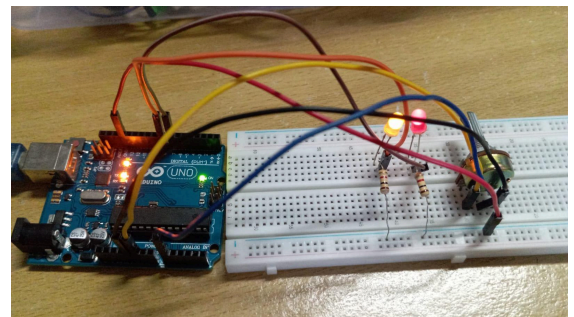
4. Pastikan total memori stack semua task tidak melebihi RAM Arduino (2KB). Dengan 128 bytes per task dan overhead FreeRTOS ~500 bytes, maksimum sekitar 10 task dapat dibuat.

→ Pertanyaan 3:

Modifikasi program dengan menambah sensor potensiometer untuk mengontrol kecepatan LED.

Pembahasan:

Modifikasi dilakukan dengan menambahkan task baru (TaskSensor) yang membaca nilai ADC dari potensiometer pada pin A0 menggunakan `analogRead()`. Nilai ADC (0-1023) dikonversi ke rentang delay (50-1000ms) menggunakan fungsi `map()`. Nilai delay ini disimpan dalam variabel global volatile yang dibaca oleh TaskBlink1 dan TaskBlink2 untuk menentukan kecepatan kedipannya. Hasilnya, memutar potensiometer ke kanan mempercepat kedipan LED, sedangkan memutar ke kiri memperlambat kedipan LED secara real-time, membuktikan kemampuan RTOS untuk merespons input sensor secara dinamis [5].



Gambar 3.5 Modifikasi Percobaan 5A: Komunikasi Task

```
#include <Arduino_FreeRTOS.h>

// PIN
const int potPin = A0;
const int led1 = 8;
const int led2 = 7;

// GLOBAL
int delayValue = 200; // default

// TASK
void TaskReadPot(void *pvParameters);
void TaskBlink1(void *pvParameters);
void TaskBlink2(void *pvParameters);

void setup() {
    Serial.begin(9600);

    pinMode(led1, OUTPUT);
    pinMode(led2, OUTPUT);

    // Create Tasks
    xTaskCreate(TaskReadPot, "ReadPot",
128, NULL, 2, NULL);
    xTaskCreate(TaskBlink1, "Blink1",
128, NULL, 1, NULL);
    xTaskCreate(TaskBlink2, "Blink2",
128, NULL, 1, NULL);
```

```

    vTaskStartScheduler();
}

void loop() {
    // kosong
}

// TASK READ POT
void TaskReadPot(void *pvParameters) {
    int potValue;

    for (;;) {
        potValue = analogRead(potPin);

        // Mapping nilai ADC (0-1023) ke
        delay (50 - 1000 ms)
        delayValue = map(potValue, 0, 1023,
        50, 1000);

        Serial.print("Pot: ");
        Serial.print(potValue);
        Serial.print(" | Delay: ");
        Serial.println(delayValue);

        vTaskDelay(200 /
        portTICK_PERIOD_MS);
    }

// TASK BLINK 1
void TaskBlink1(void *pvParameters) {
    for (;;) {
        digitalWrite(led1, HIGH);
        vTaskDelay(delayValue /
        portTICK_PERIOD_MS);

        digitalWrite(led1, LOW);
        vTaskDelay(delayValue /
        portTICK_PERIOD_MS);
    }

// TASK BLINK 2
void TaskBlink2(void *pvParameters) {
    for (;;) {
        digitalWrite(led2, HIGH);
        vTaskDelay((delayValue + 100) /
        portTICK_PERIOD_MS);

        digitalWrite(led2, LOW);
        vTaskDelay((delayValue + 100) /
        portTICK_PERIOD_MS);
    }
}

```

4.2 PERCOBAAN 5B: MULTITASKING

Percobaan 5B berhasil menampilkan data suhu dan kelembapan secara berulang pada Serial Monitor. Output menunjukkan nilai statis (temp=54, humidity=30) yang dikirim task

read_data melalui queue dan diterima oleh task display. Kedua task bekerja secara sinkron melalui mekanisme blocking queue.

Jawaban Pertanyaan Percobaan 5A

→ Pertanyaan 1:

Apakah kedua task berjalan secara bersamaan atau bergantian? Jelaskan mekanismenya!

Pembahasan:

Kedua task berjalan secara bergantian dengan mekanisme sinkronisasi queue. Task read_data mengirim data ke queue menggunakan xQueueSend() dengan parameter portMAX_DELAY, artinya task ini akan blocked jika queue penuh. Task display menerima data dengan xQueueReceive() dan juga blocked jika queue kosong. Karena kapasitas queue hanya 1 elemen, mekanisme ini menciptakan pola produsen-konsumen yang tersinkronisasi: read_data menulis, display membaca, begitu seterusnya secara bergantian [5].

→ Pertanyaan 2:

Apakah program ini berpotensi mengalami race condition? Jelaskan!

Pembahasan:

Program ini TIDAK berpotensi mengalami race condition. Hal ini karena FreeRTOS mengimplementasikan operasi queue sebagai operasi atomik yang dilindungi oleh mekanisme critical section. Ketika xQueueSend() dieksekusi, operasi tersebut tidak dapat diinterupsi oleh task lain hingga selesai. Begitu pula dengan xQueueReceive(). Berbeda jika data dibagikan melalui variabel global biasa tanpa mutex, yang sangat rentan terhadap race condition karena operasi baca-tulis tidak atomik [3]. Queue FreeRTOS dirancang khusus untuk komunikasi antar-task yang aman tanpa risiko race condition.

→ Pertanyaan 3:

Modifikasi program menggunakan sensor DHT sesungguhnya.

Pembahasan:

Modifikasi dilakukan dengan mengganti nilai statis (x.temp=54; x.h=30) dengan pembacaan sensor DHT menggunakan library DHT.h. Objek sensor DHT dideklarasikan secara global (DHT dht(DHT_PIN, DHT11)). Dalam task read_data, nilai suhu dan kelembapan dibaca menggunakan dht.readTemperature() dan dht.readHumidity(), lalu nilai tersebut dimasukkan ke struct readings sebelum dikirim ke queue. Hasilnya, Serial Monitor menampilkan nilai suhu dan kelembapan aktual lingkungan yang berubah secara dinamis, membuktikan efektivitas queue untuk akuisisi dan transmisi data sensor secara real-time [3].

```
#include <Arduino_FreeRTOS.h>
#include <queue.h>
#include <DHT.h>

// DHT CONFIG
#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// STRUCT
struct readings {
    float temp;
    float hum;
};

// QUEUE
QueueHandle_t my_queue;

// TASK
void read_data(void *pvParameters);
void display_data(void *pvParameters);

void setup() {
    Serial.begin(9600);

    dht.begin();

    // Membuat queue
    my_queue = xQueueCreate(5,
        sizeof(struct readings));

    // Membuat task
    xTaskCreate(read_data, "ReadData",
        128, NULL, 1, NULL);
```

```
xTaskCreate(display_data, "Display",
    128, NULL, 1, NULL);

vTaskStartScheduler();
}

void loop() {
    // kosong
}

// TASK READ SENSOR
void read_data(void *pvParameters) {

    struct readings sensorData;

    for (;;) {

        sensorData.temp =
        dht.readTemperature();
        sensorData.hum =
        dht.readHumidity();

        // Kirim data ke queue
        xQueueSend(my_queue, &sensorData,
            portMAX_DELAY);

        vTaskDelay(1000 /
            portTICK_PERIOD_MS);
    }

}

// TASK DISPLAY
void display_data(void *pvParameters) {

    struct readings sensorData;

    for (;;) {

        if (xQueueReceive(my_queue,
            &sensorData, portMAX_DELAY) == pdPASS)
        {

            Serial.print("Suhu : ");
            Serial.print(sensorData.temp);
            Serial.println(" C");

            Serial.print("Kelembapan : ");
            Serial.print(sensorData.hum);
            Serial.println(" %");

            Serial.println("=====");
        }
    }
}
```

4.3 PERTANYAAN PRAKTIKUM

4.3.1 JELASKAN ERBEDAAN ANTARA LOOP() PADA ARDUINO BIASA DENGAN SISTEM YANG MENGGUNAKAN RTOS

Pada Arduino biasa tanpa RTOS, fungsi loop() adalah satu-satunya 'task' yang berjalan

dalam infinite loop. Semua operasi harus dikelola manual di dalamnya secara sekuensial, sehingga jika ada operasi yang memakan waktu lama (seperti `delay()`), seluruh program terhenti menunggu. Ini disebut *cooperative/sequential execution* dan sangat tidak efisien untuk aplikasi multitasking [1].

Sebaliknya, pada sistem RTOS, fungsi `loop()` umumnya dibiarkan kosong atau tidak digunakan karena peran utamanya telah digantikan oleh task-task FreeRTOS yang dikelola kernel scheduler. Setiap task memiliki loop infinitenya sendiri (`while(1)` atau `for(;;)`) dan menggunakan `vTaskDelay()` untuk memberikan kesempatan CPU kepada task lain. Scheduler RTOS secara otomatis menggilir eksekusi task berdasarkan prioritas dan status blocking, menghasilkan sistem yang responsif dan efisien [2].

4.3.2 FUNGSI LOOP() DIBIARKAN KOSONG?

Fungsi `loop()` dibiarkan kosong karena setelah `vTaskStartScheduler()` dipanggil di `setup()`, kendali penuh diserahkan kepada FreeRTOS kernel. Kernel mengambil alih pengaturan eksekusi CPU dan tidak pernah mengembalikan kontrol ke fungsi `loop()` standar Arduino. Semua logika program yang sebelumnya ditulis di `loop()` kini dipindahkan ke dalam task-task FreeRTOS yang terdaftar [2].

Jika ada kode di dalam `loop()`, kode tersebut berpotensi mengganggu penjadwalan task RTOS atau bahkan tidak pernah dieksekusi sama sekali tergantung implementasi. Membiarkan `loop()` kosong adalah praktik standar dalam pengembangan FreeRTOS di Arduino untuk memastikan seluruh aliran eksekusi dikelola oleh kernel scheduler secara eksklusif [4].

4.3.3 APA INSIGHT UTAMA YANG ANDA PELAJARI DARI PRAKTIKUM INI?

Praktikum ini memberikan beberapa insight fundamental tentang pengembangan sistem embedded:

1. **Abstraksi dan Modularitas:** RTOS memungkinkan pemisahan logika program menjadi task-task independen yang lebih mudah dikembangkan, diuji, dan dipelihara dibandingkan satu loop monolitik.
2. **Efisiensi Waktu CPU:** Dengan `vTaskDelay()` dan mekanisme

blocking, CPU tidak pernah terbuang hanya untuk menunggu. Setiap milidetik waktu tunggu suatu task dimanfaatkan untuk menjalankan task lain, meningkatkan responsivitas sistem secara keseluruhan.

3. **Keamanan Komunikasi Data:** Queue FreeRTOS menyediakan mekanisme transfer data antar-task yang inherently thread-safe tanpa perlu penanganan mutex manual, menyederhanakan pengembangan sistem concurrent yang andal.
4. **Skalabilitas:** Menambah fitur baru (task baru) tidak memerlukan refactoring kode yang sudah ada, cukup mendaftarkan task baru melalui `xTaskCreate()`. Ini sangat menguntungkan untuk pengembangan sistem embedded yang terus berkembang.
5. **Batasan Sumber Daya:** Pengalaman langsung menunjukkan pentingnya manajemen memori yang cermat pada mikrokontroler dengan RAM terbatas (2KB pada ATmega328P), karena setiap task membutuhkan alokasi stack tersendiri [4].

5. KESIMPULAN

Praktikum Modul 5 tentang Real-Time Operating System berhasil mendemonstrasikan dua aspek penting implementasi FreeRTOS pada Arduino Uno. Percobaan 5A membuktikan kemampuan FreeRTOS dalam menjalankan tiga task secara concurrent melalui mekanisme preemptive scheduling, di mana task-task dengan prioritas sama dijadwalkan secara round-robin dan setiap task memanfaatkan `vTaskDelay()` untuk membebaskan CPU bagi task lain selama masa tunggu.

Percobaan 5B membuktikan efektivitas queue sebagai mekanisme komunikasi antar-task yang aman dari race condition. Pola produsen-konsumen antara task `read_data` dan `display` menunjukkan bahwa queue FreeRTOS mampu menyinkronisasi transfer data secara atomik tanpa risiko korupsi data. Modifikasi menggunakan sensor DHT nyata lebih lanjut memvalidasi kemampuan sistem dalam akuisisi data real-time.

Secara keseluruhan, praktikum ini membuktikan bahwa FreeRTOS merupakan solusi yang efektif dan efisien untuk mengembangkan sistem embedded multitasking bahkan pada mikrokontroler berdaya rendah, membuka potensi pengembangan aplikasi IoT dan sistem kontrol yang lebih kompleks dan andal.

DAFTAR PUSTAKA

- [1] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2023. [Online]. Available: https://www.freertos.org/Documentation/RTOS_book.html
- [2] I. Rosyidi, M. Fadli, dan A. Hidayat, "Implementasi FreeRTOS pada Arduino untuk Sistem Akuisisi Data Multi-Sensor," *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK)*, vol. 10, no. 2, hal. 345–354, 2023.
- [3] A. Nugroho dan B. Wardana, "Analisis Kinerja Queue-Based Task Communication pada FreeRTOS untuk Aplikasi IoT," *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, vol. 11, no. 3, hal. 210–219, 2022.
- [4] A. Saifullah, M. Hassan, dan C. Lu, "Memory-Efficient Multitasking with FreeRTOS on Resource-Constrained Microcontrollers," *IEEE Transactions on Embedded Computing Systems*, vol. 22, no. 4, hal. 1–12, 2023.
- [5] D. Kurniawan, S. Prabowo, dan T. Wibowo, "Perbandingan Metode Sinkronisasi Task pada FreeRTOS: Mutex, Semaphore, dan Queue," *Jurnal Ilmu Komputer dan Agri-Informatika*, vol. 10, no. 1, hal. 45–56, 2023.